PROGRAMACIÓN ORIENTADA A OBJETOS • PRIMERA ENTREGA

# Flota de robots de reparto

## UdeM

Sistema de gestión de una flota mixta de robots para el campus de la Universidad de Medellín

**Geronimo Torres Correa**

Fases 1 a 3 • Octubre 2026

`python · open-meteo · openstreetmap`

EL PROBLEMA

# Tres robots, cada uno con sus reglas

RobotRuedas

## Ruedas

Rápido y económico.
Solo edificios.

Documentos, paquetes

RobotOrugas

## Orugas

Más carga, más lento. Llega a zonas verdes.

No lleva frágiles (terreno irregular)

Dron

## Dron

Llega a azoteas. Carga liviana.

Solo vuela si el clima lo permite

El sistema registra la flota, recibe solicitudes de entrega, elige un robot compatible, calcula costo y tiempo, y lleva una bitácora.

FASE 1 • ANÁLISIS

# Lo que el sistema debe hacer

**12**

requisitos, 15 reglas de negocio y 7 supuestos propios

| ID | Requisito clave |
|---|---|
| RF1 | Registrar robots con tipo, carga máxima, velocidad, batería y ubicación |
| RF5 | Asignar la solicitud a un robot compatible (peso, carga, batería, zona) |
| RF6 | Consultar el clima (Open-Meteo) antes de asignar un dron |
| RF7 | Reasignar o dejar pendiente si no se puede asignar |
| RF8 | Calcular costo y tiempo según robot, distancia y carga |
| RF9 | Registrar eventos en una bitácora exportable |

FASE 1 • SUPUESTOS PROPIOS

# Cuándo un dron no despega

| 30 | 40 | 50% | 0 |
|---|---|---|---|
| km/h de viento sostenido | km/h en ráfagas | de probabilidad de lluvia | is_day: de noche no se vuela |

**API caída o más de 5 s:** se asume mal clima y el dron no vuela.

**Batería:** un robot debe terminar con al menos 15 %.

**Peligrosos:** nunca viajan en dron.

# Del enunciado a las tarjetas CRC

| Sustantivo | Decisión |
|---|---|
| batería | Clase Bateria: tiene reglas propias (0–100 %, reserva) |
| dron, orugas… | Subclases de Robot: cambian el comportamiento |
| velocidad | Atributo: dato simple, sin comportamiento |
| clima, API | Interfaz ProveedorClima: aislar la API |
| universidad | Fuera del modelo: es el cliente |

**AsignadorEntregas**

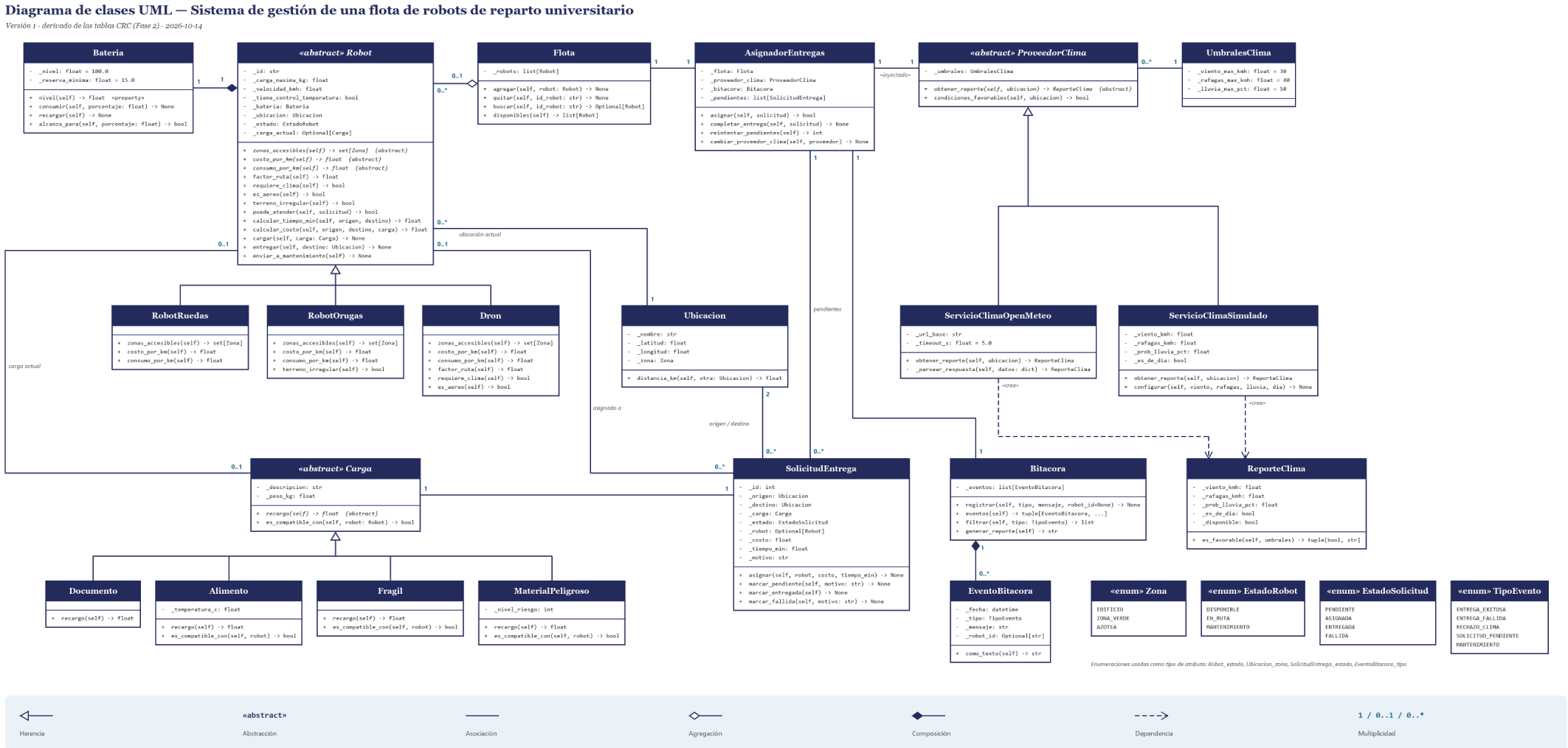
**RESPONSABILIDADES**

- Elegir el robot compatible más barato
- Consultar el clima antes de un dron
- Reasignar o dejar pendiente

**COLABORAN**

- Flota
- ProveedorClima
- Bitacora

# Diagrama de clases v1: 25 clases



FASE 3 • LO QUE EXIGE EL PRD

# Cuatro relaciones con su razón

### Herencia + polimorfismo

Robot → Ruedas, Orugas, Dron. Solo el Dron responde sí a requiere_clima().

### Composición

Robot ◆ Bateria: la batería nace y muere con su robot.

### Agregación

Flota ◇ Robot: un robot puede salir de la flota y seguir existiendo.

### Clima desacoplado

ProveedorClima se inyecta en AsignadorEntregas. Asociación, nunca herencia.

TRAZABILIDAD ENTRE FASES

# Cada clase viene de un requisito

| Análisis (Fase 1) | Tarjeta CRC (Fase 2) | UML v1 (Fase 3) |
|---|---|---|
| RF6 Clima antes del dron | ProveedorClima | condiciones_favorables() • Dron.requiere_clima() |
| RF7 Reasignar o pendiente | AsignadorEntregas | _pendientes • marcar_pendiente() |
| RF8 Costo y tiempo | Robot (polimórfico) | costo_por_km() en cada subtipo • recargo() |
| S1 Umbral 30 / 40 / 50 % | UmbralesClima | UmbralesClima con los tres límites |
| S3 Reserva de batería 15 % | Bateria | _reserva_minima • alcanza_para() |

**El modelo evolucionó:** 7 cambios del CRC al UML, cada uno con su motivo, registrados en CHANGELOG_UML.md.

API METEOROLOGICA

# Open-Meteo decide si el dron vuela

Gratis, sin registro ni API key. Una petición con las coordenadas del campus devuelve:

| | |
|---|---|
| `wind_speed_10m` | más de 30 km/h: no vuela |
| `wind_gusts_10m` | más de 40 km/h: no vuela |
| `precipitation_probability` | más de 50 %: no vuela |
| `is_day` | 0 (noche): no vuela |

## ¿Y si la API falla?

Timeout de 5 s, error HTTP o JSON inválido: el reporte vuelve como "no disponible".

El dron no vuela, se registra en la bitácora y la entrega pasa a un robot terrestre o queda pendiente.

API PARA LA INTERFAZ GRÁFICA

# OpenStreetMap dibuja el campus real

### Mapa real de la UdeM

Los bloques, canchas y azoteas ya están mapeados con sus coordenadas.

### Gratis y sin API key

Se usa desde Python con tkintermapview. Solo pide mostrar la atribución.

### Solo visualización

Vive en la capa de interfaz. No toca las reglas de asignación.

### Si falla, nada se rompe

Sin mapa, el sistema sigue asignando entregas igual.

APIS OPCIONALES • POR CONFIRMAR

# Extras, cada uno con respaldo

| API | Qué aportaría | Riesgo | Si falla |
|---|---|---|---|
| OSRM | Distancia real a pie entre bloques para robots terrestres | Servidor demo comunitario, sin garantía | Línea recta (Haversine) × 1,3 |
| Nager.Date | Festivos: la cafetería cierra y no hay entregas de alimentos | Regla de negocio extra que hay que validar | Se ignora la regla de festivos |

Ambas irían detrás de su propia interfaz, con el **mismo patrón que el clima**: una implementación real y una de respaldo.

VISTA PREVIA • CÓDIGO (PYTHON)

# Cambiar el proveedor es una línea

```
# el asignador recibe el clima, no lo crea
asignador = AsignadorEntregas(
    flota,
    ServicioClimaOpenMeteo(umbrales),
    bitacora)

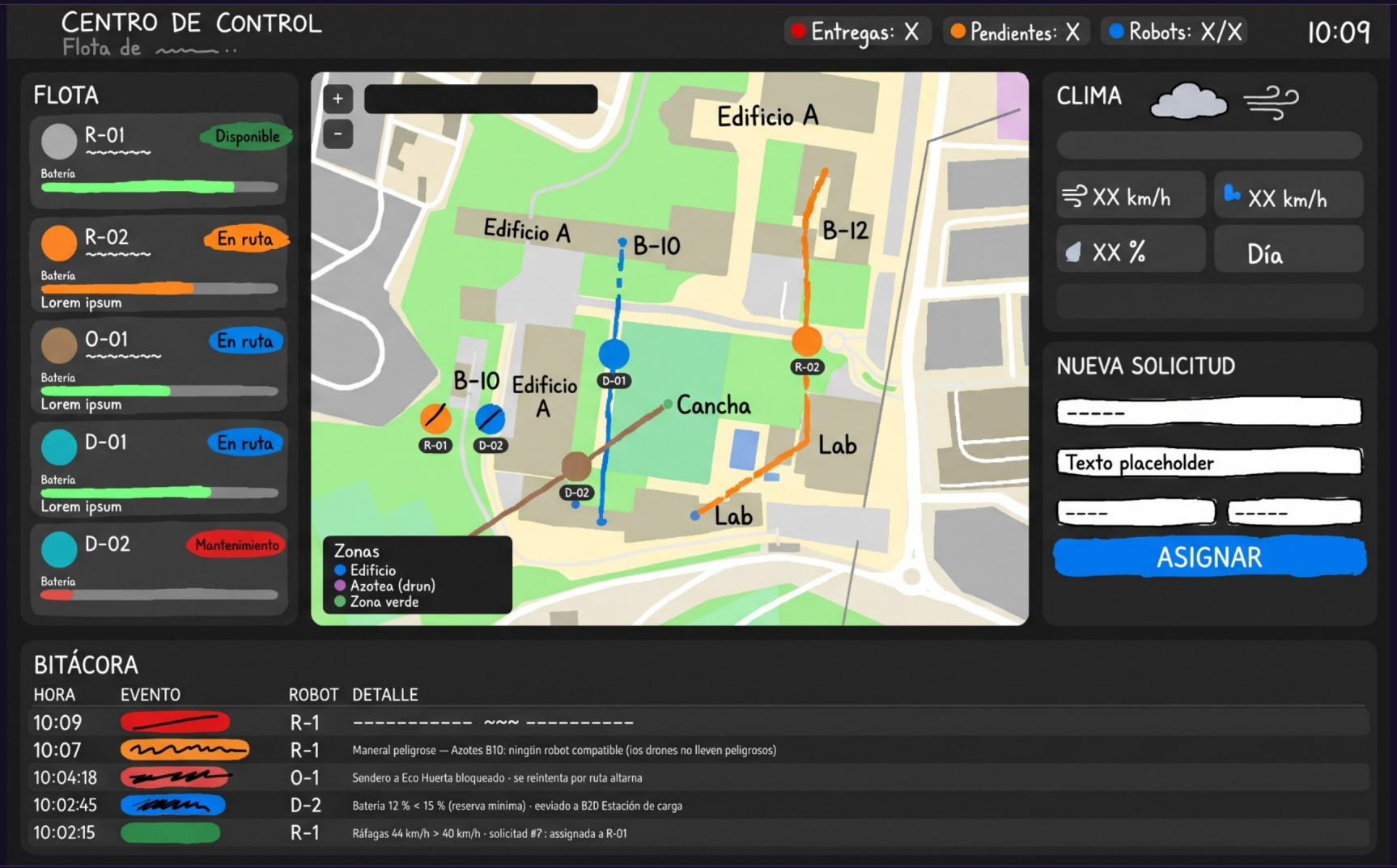
# prueba o defensa: otro proveedor
asignador.cambiar_proveedor_clima(
    ServicioClimaSimulado(umbrales, viento=45))
```

**AsignadorEntregas** solo conoce la interfaz **ProveedorClima**.

No importa requests ni sabe de URLs o JSON. Por eso la prueba de cambiar el proveedor en vivo no toca la lógica de asignación.

VISTA PREVIA • INTERFAZ

# Boceto del centro de control




Flota • mapa del campus • clima con interruptor real/simulado • nueva solicitud • bitácora • 13

PRÓXIMOS PASOS

# Del diseño al código

día 8

## Fase 4

Justificación: alternativas y principio de cada decisión

días 9–12

## Fase 5

Dominio, clima, asignador, interfaz y tests: 10 commits más

día 13

## Defensa

Demo y cambio de proveedor de clima en vivo

día 14

## Fase 6

Reflexión individual y entrega final

github.com/torreto28/proyecto-poo